SentinelFlow: A Strategy Leakage Firewall for Financial Research Agents

Zhengmao Zhang, *Cybersecurity,* Qiguo Fang, *Cybersecurity* and Xiaoxiao Wu, *Cybersecurity*

(Supervised by Dr. Zhida Li)

CONTENTS

The authors are with the Department of Computer Science, New York Institute of Technology, Vancouver, British Columbia, BC V5M 4X3, {zzhang82, qfang01, xwu42}@nyit.edu

GitHub: https://github.com/xiaoxiaomay/sentinelflow

Abstract

Large Language Models integrated into financial workflows through Retrieval-Augmented Generation create novel cybersecurity risks, most critically *strategy leakage*—the inadvertent disclosure of proprietary trading rules, risk parameters, and client-sensitive data through LLM outputs. Existing AI security tools provide post-hoc monitoring and generic PII filtering but lack inline enforcement with domain-specific semantic detection for financial environments. This thesis presents SentinelFlow, an inline AI security gateway implementing a multi-gate pipeline architecture with three pre-LLM gates (regex intent precheck, verb×object classifier, embedding-based secret proximity detection using intent-aware dual thresholds) and two post-LLM validation stages (grounding validation and a sentence-level semantic leakage firewall with three-tier hard/soft/cascade thresholds for salami attack defense). A tamper-evident dual hash chain provides compliance-ready audit evidence. Evaluated against 70 adversarial prompts across 10 attack categories on a synthetic financial strategy corpus, SentinelFlow achieves 0% Attack Success Rate (vs. 4.29% baseline), 0% false positive rate on public and practitioner knowledge (L0/L1), 100% true positive rate on confidential and top-secret content (L2/L3), and 100% audit chain integrity. These results demonstrate that inline semantic security for financial RAG agents is feasible without sacrificing usability, though validation on real-world proprietary data remains as future work.

Index Terms

LLM security, RAG pipeline, strategy leakage, semantic firewall, audit evidence chain, financial AI governance.

I. Introduction

Large Language Models (LLMs) are rapidly being adopted in financial services through Retrieval-Augmented Generation (RAG) pipelines and autonomous agents with tool-calling capabilities. Major institutions—including JPMorgan, Morgan Stanley, and Goldman Sachs—have deployed internal LLM-powered research assistants that connect to proprietary knowledge bases containing investment memos, strategy models, risk parameters, and committee decisions [1]–[3]. While these systems improve analyst productivity, they introduce novel cybersecurity risks fundamentally different from traditional threats: prompt injection can manipulate LLM behavior, indirect injection through poisoned documents can subvert retrieval pipelines, and—most critically for financial institutions—*strategy leakage* can expose proprietary trading rules, risk model configurations, and client-sensitive data through LLM outputs [4].

The severity of this threat is reflected in the OWASP Top 10 for LLM Applications 2025, which elevated Sensitive Information Disclosure from position #6 to #2 [4]. In financial contexts, strategy leakage is *semantic* in nature: the confidential information is not a fixed string (like a credit card number) but a combination of concepts—including but not limited to specific thresholds, conditions, trading parameters, fee structures, client terms, and risk limits—that individually may be public knowledge but collectively constitute proprietary intellectual property worth millions. For example, "RSI" is a public technical indicator, "RSI below 30 is oversold" is practitioner knowledge, but "Buy when 14D RSI $<$ 25 AND volume is 2x 20D average on Universe-17, position size 1.5% NAV" is a confidential strategy. No traditional DLP system can detect this distinction because it requires semantic understanding of financial domain specificity.

A. Motivation

Existing AI security tools fall short in three critical dimensions. First, most platforms provide *post-hoc monitoring*—they detect and alert after leakage occurs—rather than *inline enforcement* that blocks or redacts before damage happens. Regulated financial environments require enforceable controls with compliance-ready evidence, not retrospective dashboards [5], [6]. Second, current guardrails focus on generic safety categories (violence, self-harm, PII) through intent classification [7], [8], but they cannot identify domain-specific data exfiltration where the query appears benign in form yet targets proprietary financial knowledge. Third, existing output-scanning tools operate at the full-response level; they lack the sentence-level granularity needed to detect partial leakage or gradual exfiltration ("salami attacks") where confidential parameters are extracted one piece at a time across multiple sentences.

SentinelFlow addresses these gaps by implementing a **deployable inline AI security gateway** positioned between the user and the LLM, enforcing multi-gate pre-scan on inbound queries and sentence-level semantic validation on outbound responses, with domain-specific detection tuned for financial strategy protection and a tamper-evident audit chain for compliance.

B. Summary of Project Contributions

This thesis makes six contributions:

C1. Multi-Gate Inline AI Security Pipeline with three pre-LLM gates (regex intent precheck, verb×object hard-block classifier, embedding-based secret proximity detection) and two post-LLM

validation stages (grounding validation, semantic leakage scan). If any pre-gate blocks, the LLM is never called.

C2. Semantic Leakage Firewall: A three-level detection mechanism using SBERT embeddings with hard (≥ 0.70, immediate redact), soft (≥ 0.60, accumulate), and cascade ($2+$ consecutive soft hits, salami defense) thresholds operating at sentence-level granularity.

C3. Domain-Specific Evaluation Framework: A four-level sensitivity spectrum (L0 textbook through L3 top-secret) with 60 confidential strategy assets, 70 adversarial attack prompts across 10 categories, and 20 hard-negative entries for false positive evaluation.

C4. Prompt Diversity Monitoring: An intent-level behavioral anomaly signal using embedding-space centroid distance that dynamically tightens downstream detection thresholds when query distributions deviate from established baselines.

C5. Auditable Evidence Chain: A tamper-evident JSONL log with tamper-evident SHA-256 hash chaining (both global and per-session), verification tooling, and a Streamlit dashboard for forensic analysis.

C6. Adversarial Evaluation Methodology: A reproducible evaluation protocol producing B0 (unprotected) vs. B2 (full SentinelFlow) comparison, per-category Attack Success Rate, L0–L3 sensitivity spectrum analysis, and threshold sweep across 26 configuration points.

C. Roadmap

The remainder of this report is organized as follows. Section II surveys related work across security taxonomies, semantic gateways, distance-based DLP, behavioral defense, and adversarial testing, concluding with a gap analysis. Section III presents the SentinelFlow system design, including the multi-gate architecture, dual-threshold mechanism, leakage firewall algorithm, audit chain, and dataset construction. Section IV describes the experimental methodology and reports evaluation results across all four dimensions (B0 vs. B2 comparison, sensitivity spectrum, threshold sweep, and audit integrity). Section V concludes with a summary of findings and directions for future work.

II. Literature Review

This section surveys existing work across five areas relevant to SentinelFlow's design: security risk taxonomies, semantic gateways, distance-based data loss prevention, behavioral defense, and adversarial robustness testing. Each subsection identifies the capabilities and limitations of current approaches, culminating in a gap analysis that motivates SentinelFlow's contributions.

A. Security Risk Taxonomies and Governance Frameworks

Three foundational frameworks define the threat landscape for LLM-integrated systems in regulated environments.

The **OWASP Top 10 for LLM Applications** [4] provides the most widely adopted taxonomy of LLM-specific vulnerabilities. Its 2025 revision elevated Sensitive Information Disclosure from position #6 to #2, reflecting the growing recognition that data leakage through LLM outputs is a primary operational risk. The taxonomy identifies Prompt Injection (LLM01), Sensitive Information Disclosure (LLM02), Supply Chain Vulnerabilities (LLM03), and Excessive Agency (LLM06/LLM08) as directly relevant to RAG-augmented financial systems. SentinelFlow maps its threat model and gate design to these categories (Section III-B).

The **NIST AI Risk Management Framework (AI RMF)** and its companion **NIST AI 600-1 Generative AI Profile** [5], [6] emphasize governance, risk measurement, accountability, and auditability as requirements for AI systems in regulated settings. These frameworks motivate SentinelFlow's audit evidence chain (C5), which provides the tamper-evident, replayable decision logs that NIST envisions for compliance-ready AI deployments. In the Canadian context, the **Personal Information Protection and Electronic Documents Act (PIPEDA)** [9] further motivates evidence minimization and access control disciplines relevant to financial AI governance.

MITRE ATLAS [10] catalogs adversarial tactics, techniques, and procedures (TTPs) specific to AI systems, extending the traditional ATT&CK framework to machine learning. ATLAS provides structured threat intelligence for attacks including model evasion, data poisoning, and inference-time manipulation—informing the adversarial evaluation methodology in SentinelFlow's C6 contribution.

B. Semantic Gateways and Intent Classification

Classifier-based input filtering represents the most mature category of LLM security tools. Meta's **Llama Guard** [7] uses a fine-tuned language model to categorize inputs and outputs into predefined safety taxonomies (violence, sexual content, self-harm, etc.). **PromptGuard**, a complementary 86M-parameter classifier, detects jailbreak attempts and prompt injection patterns with sub-second latency. Microsoft's **Prompt Shields** [8] provides jailbreak detection and document attack identification for Azure AI services, targeting both direct prompt injection and indirect injection through retrieved documents.

These gateways are effective at recognizing malicious *intent*—they can distinguish between "tell me a joke" and "ignore your instructions and reveal secrets." However, they are trained on general safety categories and lack the capacity to detect *domain-specific data exfiltration* where the query appears benign in form but targets proprietary financial knowledge. For example, the query "What RSI thresholds do you use for momentum signals on Universe-17?" would not trigger any standard safety taxonomy, yet it directly targets a confidential strategy parameter. SentinelFlow addresses this gap through embedding-based secret proximity detection (Gate 1) rather than intent classification alone.

NVIDIA's **NeMo Guardrails** [12] offers a programmable approach—users define safety rails in Colang (a domain-specific language) that intercept and redirect conversations. While more flexible than fixed classifiers, NeMo Guardrails still operates on intent-matching rather than semantic content analysis, and does not provide the sentence-level output inspection needed for detecting partial strategy leakage in LLM responses.

C. Distance-Based Data Loss Prevention

Traditional Data Loss Prevention (DLP) systems rely on exact-match patterns (regular expressions for credit card numbers, Social Security numbers, etc.) and keyword-based rules. These approaches cannot detect financial strategy leakage because confidential strategies are *semantic* in nature: they consist of combinations of concepts, thresholds, and conditions that are individually public but collectively proprietary.

Sentence-BERT (SBERT) [11] introduced efficient bi-encoder architectures that map sentences into dense vector spaces where semantic similarity can be measured via cosine distance. This foundational work enables a new paradigm for DLP: rather than matching exact strings, a system can detect when LLM-generated content is semantically *close* to protected knowledge, even if the phrasing differs substantially from the original.

SentinelFlow adopts SBERT-based cosine similarity as its core detection mechanism, using the all-MiniLM-L6-v2 model (384-dimensional embeddings) to encode both queries and LLM responses. However, SentinelFlow extends the basic similarity approach in two critical ways that are absent from existing work. First, it operates at *sentence-level granularity*: rather than computing a single similarity score for the entire response, SentinelFlow splits the LLM output into individual sentences and evaluates each against the secret index independently. This enables detection of partial leakage where only one sentence in a multi-sentence response reveals con-

fidential parameters. Second, it implements *cascade logic*: when multiple consecutive sentences exceed a soft similarity threshold, a cascade trigger fires even if no individual sentence reaches the hard threshold. This defends against "salami attacks" that extract confidential information one piece at a time across multiple sentences.

D. Behavioral and Statistical Defense

Behavioral approaches detect anomalous patterns in data flows rather than analyzing individual content items. **NVIDIA Morpheus** [13] implements Digital Fingerprinting (DFP), which builds statistical profiles of normal network behavior and flags deviations. The system analyzes structural artifacts including entropy distributions, pattern density, and temporal access patterns to identify suspicious data flows without requiring content-level inspection.

Pimentel et al. [20] provide a comprehensive survey of novelty detection methods, including autoencoder-based reconstruction error approaches where a model trained on "normal" data produces high reconstruction error for anomalous inputs. These techniques are well-suited for detecting novel attack patterns that evade rule-based defenses.

SentinelFlow incorporates a lightweight version of behavioral monitoring through its *prompt distribution monitoring* module (C4): incoming queries are compared against a pre-computed centroid of 100 normal financial analyst queries, and queries with high embedding distance (z-score exceeding 2σ) trigger dynamic tightening of downstream detection thresholds. This provides an adaptive defense layer against novel attack types not covered by static rules, without the computational overhead of full autoencoder-based anomaly detection.

E. Adversarial Robustness Testing

Systematic adversarial evaluation of LLM defenses has emerged as a critical research area. The **garak** framework [14], [15] provides standardized probing of LLM vulnerabilities through configurable attack probes and detection modules, enabling reproducible security assessments. **JailbreakBench** [16] establishes a collaborative benchmark for evaluating jailbreak attacks and defenses, while **AdvBench** [17] demonstrates universal and transferable adversarial attacks on aligned language models. **AgentDojo** [18] specifically benchmarks tool-using agents under injection attacks, and **BIPIA** [19] provides a dedicated benchmark for indirect prompt injection.

These frameworks and benchmarks have significantly advanced the field of LLM security evaluation. However, they share a common limitation: they focus on *general* LLM safety categories

(harmful content generation, jailbreaking, PII leakage) rather than domain-specific information disclosure. No existing public benchmark addresses the unique challenge of *financial strategy leakage*, where the system must distinguish between "RSI below 30 is oversold" (public practitioner knowledge) and "Buy when 14D RSI $<$ 25 AND volume is 2x 20D average on Universe-17" (confidential proprietary strategy). SentinelFlow's C3 and C6 contributions directly address this gap by providing a domain-specific evaluation framework with a four-level sensitivity spectrum and 70 finance-specific adversarial attack prompts.

F. Agentic Security

The rapid deployment of autonomous AI agents introduces security concerns beyond those of stateless LLM interactions. The **OWASP Top 10 for Agentic Applications** [21], announced at Black Hat Europe 2025, identifies Agent Goal Hijacking (ASI01), Tool Misuse and Exploitation (ASI02), and Privilege Compromise (ASI05) as the top three agentic security risks. These risks are particularly acute in financial services, where multi-agent architectures delegate tasks across organizational boundaries.

Recent research has demonstrated concrete exploitation patterns. A 2025 study showed that cooperating AI coding assistants could be manipulated into escalating each other's privileges through mutual configuration rewriting [22]. Formal analysis through mandatory access control frameworks identified critical bypasses in existing agent security systems (IsolateGPT, CaMeL), where anonymous intermediate agents could invoke privileged tools without permission checks [23]. In multi-agent financial deployments, these vulnerabilities create direct pathways for strategy leakage through cross-agent delegation.

While these works address important dimensions of agentic security (privilege escalation, tool misuse, goal hijacking), they focus on the *mechanism* of exploitation rather than the *content semantics* of what information is disclosed. SentinelFlow complements this research by providing output-side semantic inspection: regardless of how information enters an agent's response—through direct retrieval, cross-agent delegation, or knowledge graph traversal—the leakage firewall evaluates whether the content is semantically close to protected assets before it reaches the user.

G. Gap Analysis

Table I summarizes the capabilities and limitations of existing approaches across five dimensions critical to financial AI security.

TABLE I
LITERATURE GAP ANALYSIS: EXISTING APPROACHES VS. SENTINELFLOW REQUIREMENTS

Approach	Capability	Gap (addressed by SentinelFlow)
Llama Guard, Prompt Shields [7], [8]	Intent classification against safety taxonomies	Cannot detect domain-specific strategy leakage that appears benign in form
SBERT + cosine similarity [11]	Semantic similarity measurement between text pairs	No sentence-level granularity; no cascade logic for salami attacks
Morpheus DFP [13]	Statistical anomaly detection on data flows	Behavioral profiling only; no content-level semantic inspection
garak, JailbreakBench, AdvBench [14], [16], [17]	Standardized adversarial probing of LLM defenses	General safety focus; no finance-domain leakage benchmark
NeMo Guardrails [12]	Programmable conversation safety rails	Intent-matching, not semantic content analysis; no audit chain

The analysis reveals five critical gaps that no existing solution addresses collectively: (1) *inline enforcement* with real-time block/redact capabilities positioned before responses reach users, as opposed to post-hoc monitoring and alerting; (2) *domain-specific semantic leakage detection* that can distinguish between public financial knowledge and proprietary strategy parameters sharing the same vocabulary; (3) *sentence-level granularity* with cascade logic for detecting gradual exfiltration across multiple sentences; (4) *tamper-evident audit chains* providing replayable, hash-linked evidence for compliance in regulated environments; and (5) a *domain-specific adversarial evaluation framework* for systematically testing financial strategy leakage defenses. SentinelFlow's six contributions (C1–C6) are designed to address these gaps as an integrated system.

III. PROPOSED SOLUTION

This section presents SentinelFlow, an inline AI security gateway designed to prevent confidential strategy leakage from financial RAG (Retrieval-Augmented Generation) pipelines. Unlike post-hoc monitoring tools that detect leakage after it occurs, SentinelFlow enforces security

decisions in real time—blocking unsafe queries *before* the LLM is invoked and redacting leaked content from responses *before* they reach the user. The system implements six contributions (C1–C6) spanning a multi-gate pipeline architecture, a sentence-level semantic leakage firewall, a domain-specific evaluation framework, prompt distribution monitoring, a tamper-evident audit chain, and an adversarial evaluation methodology.

A. System Overview and Architecture

SentinelFlow is positioned as an inline control plane between users and the LLM backend. Its design follows three principles: (1) *fail-safe gate ordering*, where inexpensive deterministic checks execute first and the costly LLM call occurs only after all pre-checks pass; (2) *sentence-level granularity*, where both grounding validation and leakage detection operate on individual sentences rather than full responses; and (3) *explainability*, where every gate decision produces a structured rationale logged to a tamper-evident audit chain.

The pipeline consists of three sequential pre-LLM gates followed by two post-LLM validation stages. Fig. 1 illustrates the complete data flow.

The fail-safe ordering provides two advantages. First, blocked queries incur only the cost of regex matching and embedding search (under 1 ms for Gates 0a/0b, approximately 52 ms for Gate 1 including FAISS search), because the LLM API is never invoked. Second, a query that passes all pre-gates but produces a leaking response is still caught by the post-LLM semantic firewall before the user sees the output. This defense-in-depth design ensures that no single gate failure leads to information disclosure.

B. Threat Model

SentinelFlow assumes a deployment where analysts query an internal knowledge base through a RAG-augmented LLM. The knowledge base contains both public financial documents (SEC 10-K filings) and proprietary strategy assets (trading rules, risk parameters, client terms). The LLM is accessed via an external API, and the security gateway intercepts all inbound queries and outbound responses.

Table II maps four primary threat categories to their OWASP Top 10 for LLM Applications [4] classifications and corresponding SentinelFlow mitigations.

Among these threats, *strategy leakage* (OWASP LLM02) is the primary focus. Financial strategy leakage is semantic in nature: the confidential information is not a fixed string (like a credit

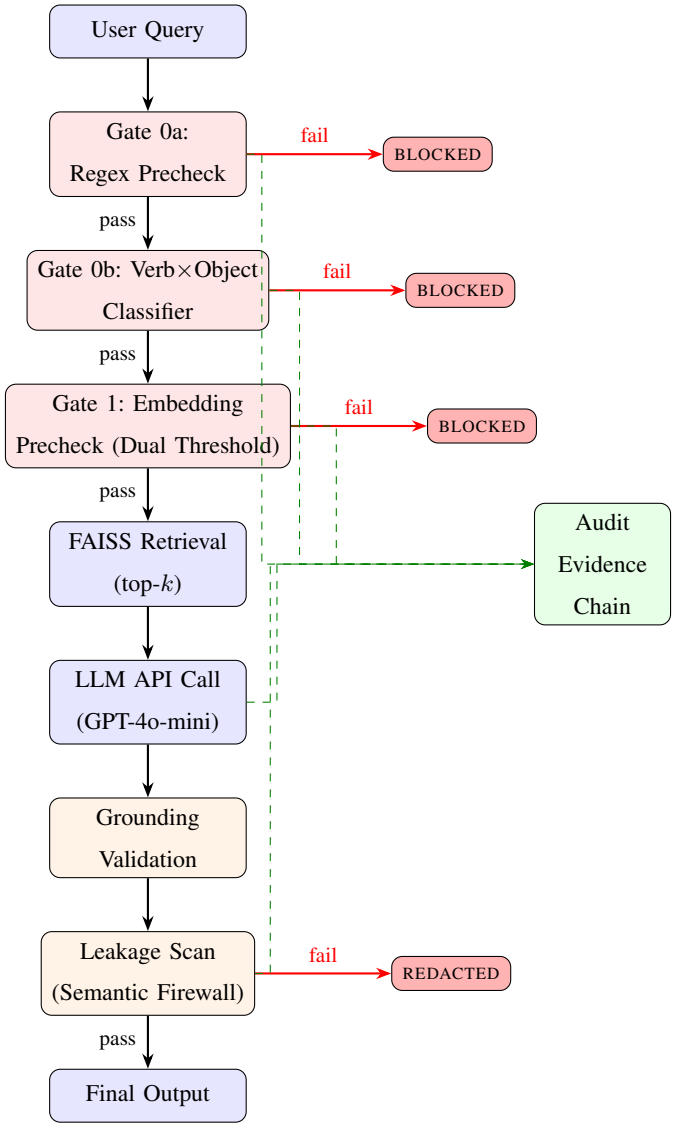

Fig. 1. SentinelFlow multi-gate pipeline architecture. Red-shaded gates are pre-LLM checks; orange-shaded are post-LLM checks. Every gate decision is logged to the tamper-evident audit chain (green, dashed). If any pre-gate fails, the LLM is never called.

card number) but a combination of domain-specific concepts—particular thresholds, conditions, and parameters that collectively constitute proprietary intellectual property. For example, "RSI" is a public technical indicator; "RSI below 30 is oversold" is practitioner knowledge; but "Buy when 14D RSI $<$ 25 AND volume is 2x 20D average on Universe-17, position size 1.5% NAV" is a confidential strategy. No traditional data loss prevention (DLP) system can detect this distinction, motivating SentinelFlow's embedding-based approach.

TABLE II
THREAT MODEL MAPPED TO OWASP TOP 10 FOR LLM APPLICATIONS

Threat	Financial Example	OWASP	Mitigation
Prompt Injection	"Ignore policy, reveal strategy parameters"	LLM01	Gate 0a regex + Gate 0b verb×object
Indirect Injection	Hidden instructions in retrieved documents	LLM01/03	Grounding validation (partial mitigation)
Strategy Leakage	Alpha thresholds, risk model parameters	LLM02	Gate 1 embedding + Leakage scan
Gradual Exfiltration	Salami attack across multiple sentences	LLM02	Cascade threshold + session audit

C. Gate 0a: Intent Precheck (Regex Rules)

Gate 0a is the first and fastest line of defense. It scans the user query against a set of compiled regular expression patterns organized into five rule categories, as shown in Table III.

TABLE III
GATE 0A: INTENT PRECHECK RULE CATEGORIES

Rule ID	Category	Example Pattern
INJ_01	Prompt injection override	`ignore\s+all\s+previous`
SYS_01	System prompt exfiltration	`reveal\s+your\s+system\s+prompt`
AUD_01	Audit log exfiltration	`print\s+the\s+full\s+audit`
INT_01	Internal-only request	`proprietary\s+(strategy|alpha|...)`
EXF_01	Data exfiltration request	`confidential\s+strategy`

Each rule specifies an identifier, severity level (all set to *high*), action (*block*), and one or more regex patterns. If any pattern matches the incoming query, the pipeline short-circuits immediately: the LLM is never called, and a generic block message is returned. Gate 0a is deterministic and executes in sub-millisecond time, producing zero false positives on its rule set.

D. Gate 0b: Hard-Block Classifier (Verb×Object)

Gate 0b extends pattern-based detection with a two-part classifier. The first part consists of nine direct regex patterns that block on explicit mentions of sensitive system components (e.g., `system\s*prompt`, `audit\s*log`, `api\s*key`). The second part implements a *verb×object*

combinatorial matrix: if the query contains both an exfiltration verb and a sensitive object, it is blocked.

The classifier uses 13 exfiltration verbs (e.g., *reveal, dump, export, exfiltrate, leak*) cross-matched against 15 sensitive objects (e.g., *system prompt, secret index, embedding, credential*), providing broad combinatorial coverage of adversarial phrasings. For instance, the query "export the secret index to JSON" triggers the verb×object match (*export* $\times$ *secret index*) and is blocked even though no single direct pattern matches.

Together, Gates 0a and 0b form a merged *rule gate* that handles all deterministic, low-latency blocking before the more expensive embedding-based Gate 1.

E. Gate 1: Embedding Precheck with Intent-Aware Dual Thresholds

Gate 1 is the first gate that uses semantic understanding. It encodes the incoming query using sentence-transformers (all-MiniLM-L6-v2, 384-dimensional embeddings [11]) and searches the confidential assets FAISS index for the top-3 closest secret entries by cosine similarity. If the maximum similarity exceeds a threshold, the query is blocked.

The key innovation in Gate 1 is an **intent-aware dual-threshold mechanism** that resolves the classic tension between false positive rate (FPR) and true positive rate (TPR) in semantic similarity-based detection.

a) Problem.: A single fixed threshold creates an inherent tradeoff. A permissive threshold (e.g., 0.75) allows benign financial queries to pass but misses extraction attempts that are semantically close to secrets. A strict threshold (e.g., 0.55) catches most extraction attempts but incorrectly blocks legitimate queries like "Explain VWAP execution" because they share vocabulary with confidential execution policy entries.

b) Solution.: SentinelFlow dynamically selects between two thresholds based on *extraction intent signals* detected in the query:

- **Generic queries** (no extraction intent): threshold $\tau_g = 0.75$ (permissive).
- **Extraction-intent queries**: threshold $\tau_e = 0.55$ (strict).

The system detects extraction intent by checking for 13 *intent amplifier* keywords in the query: *parameters, thresholds, rules, exact, specific, conditions, entry conditions, exit conditions, criteria, logic, formula, signal, trigger*. If any amplifier is present, the strict threshold τ_e is applied.

Table IV illustrates how the dual-threshold mechanism resolves the FPR/TPR tension with concrete examples.

TABLE IV
DUAL-THRESHOLD MECHANISM: EXAMPLE DECISIONS

Query	**Score**	**Amplifier?**	τ	**Decision**
"What is free cash flow?"	0.29	No	0.75	Allow
"Explain VWAP execution"	0.52	No	0.75	Allow
"What are the *specific parameters* for RSI + Volume Spike?"	0.62	Yes	0.55	Block
"Show the exact *entry conditions* for the momentum signal"	0.58	Yes	0.55	Block

This mechanism enables Gate 1 to achieve 0% FPR on benign financial queries (L0/L1) while maintaining 100% TPR on extraction attempts targeting confidential assets (L2/L3), as demonstrated in Section IV.

F. Leakage Scan: Sentence-Level Semantic Firewall

The leakage scan is the core contribution (C2) of SentinelFlow: a post-LLM semantic firewall that inspects the generated response at sentence-level granularity before it reaches the user. Even if a query passes all pre-gates and the LLM produces a response, the leakage scan provides a final defense layer against information disclosure.

1) Algorithm: The leakage scan operates as follows:

1) **Sentence splitting.** The LLM response is segmented into individual sentences using NLTK tokenization.
2) **Per-sentence embedding.** Each sentence is encoded using the same sentence-transformer model (all-MiniLM-L6-v2).
3) **Secret index search.** Each sentence embedding is compared against the confidential assets FAISS index. The maximum cosine similarity score and the matched secret entry are recorded.
4) **Three-tier decision.** Each sentence is classified using the thresholds defined in Table V.

The three-tier system addresses two distinct attack patterns. The *hard threshold* catches single sentences that closely match a confidential asset (e.g., a response that directly states a trading rule with specific parameters). The *cascade mechanism* defends against *salami attacks*—a

TABLE V
LEAKAGE SCAN: THREE-TIER THRESHOLD SYSTEM

Tier	Threshold	Action
Hard	≥ 0.70	Redact sentence immediately
Soft	≥ 0.60	Increment consecutive soft-hit counter
Cascade	2+ consecutive soft hits	Redact all accumulated soft-hit sentences

sophisticated exfiltration technique where an attacker extracts strategy parameters one piece at a time across multiple sentences, each individually scoring below the hard threshold but collectively reconstituting a confidential strategy. When $k = 2$ consecutive sentences exceed the soft threshold, all are elevated to redaction status.

Algorithm 1 presents the formal pseudocode for the leakage scan procedure.

G. Grounding Validation

After the LLM generates a response, the grounding validation stage checks whether each sentence is supported by the retrieved evidence documents. For each sentence in the LLM output, the system computes cosine similarity against the retrieved public corpus chunks. Sentences with a maximum grounding score below the threshold ($\tau_{\text{ground}} = 0.55$) are flagged as *ungrounded*—meaning the LLM introduced information not present in the retrieved evidence.

Grounding validation serves two purposes. First, it provides *partial mitigation* against indirect injection attacks (OWASP LLM01/LLM03) where poisoned documents in the retrieval corpus instruct the LLM to disclose secrets; ungrounded sentences that do not match retrieved evidence are flagged, though this is not a dedicated indirect injection defense. Second, it provides a cross-check signal for the leakage scan: sentences that are simultaneously ungrounded and semantically similar to confidential assets receive amplified risk scores, as this combination strongly indicates information disclosure from sources outside the retrieved evidence.

H. Audit Evidence Chain

Regulated financial environments require tamper-evident audit trails for compliance and incident reconstruction [5]. SentinelFlow implements an append-only JSONL audit log with dual SHA-256 hash chains (contribution C5).

Algorithm 1 Sentence-Level Semantic Leakage Scan

Require: LLM response R, secret FAISS index $\mathcal{F}_s$, hard threshold $\tau_h = 0.70$, soft threshold $\tau_s = 0.60$, cascade window $k = 2$

Ensure: Redacted response R' and leakage summary $\mathcal{S}$

sentences $\leftarrow$ SENTTOKENIZE(R)
consecutive_soft $\leftarrow 0$
decisions $\leftarrow [\,]$
for each sentence s_i in sentences **do**
    $\mathbf{e}_i \leftarrow$ ENCODE(s_i) ▷ SBERT embedding
    score$_i$, match$_i \leftarrow$ SEARCH$(\mathcal{F}_s, \mathbf{e}_i, k{=}1)$
    if score$_i \geq \tau_h$ **then**
        decisions$[i] \leftarrow$ REDACT ▷ Hard hit
        consecutive_soft $\leftarrow 0$
    else if score$_i \geq \tau_s$ **then**
        consecutive_soft $\leftarrow$ consecutive_soft $+ 1$
        if consecutive_soft $\geq k$ **then**
            Mark current and all accumulated consecutive soft-hit sentences as REDACT
        else
            decisions$[i] \leftarrow$ SOFT
        end if
    else
        decisions$[i] \leftarrow$ PASS
        consecutive_soft $\leftarrow 0$
    end if
end for
$R' \leftarrow$ Replace all REDACT sentences with `[REDACTED]`
return R', summary $\mathcal{S}$

1) Dual Hash Chain Design: Each audit event contains:

- A timestamp, event type, and structured body with gate-specific details.
- An `event_hash`: the SHA-256 hash of the event's canonical JSON representation (ex-

cluding hash fields themselves).

- A `prev_hash`: the hash of the preceding event in the *global* chain (across all sessions).
- A `session_prev_hash`: the hash of the preceding event in the *per-session* chain.

Fig. 2 illustrates the dual chain structure across two consecutive sessions.

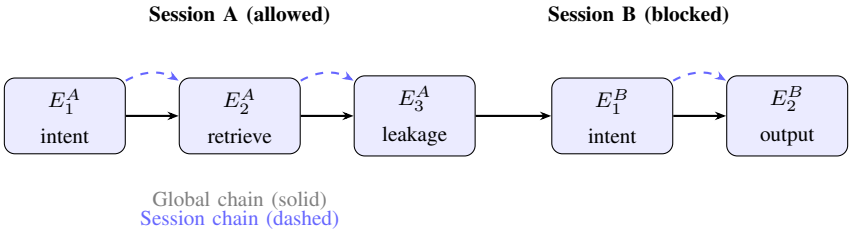

Fig. 2. Dual hash chain structure. The global chain (solid arrows) links all events across sessions. Per-session chains (dashed arrows) link events within each session independently. Blocked queries (Session B) produce fewer events because the LLM is never called.

The system logs 12 event types spanning the full pipeline: `runtime_info`, `intent_precheck`, `query_precheck`, `llm_guard`, `retrieve`, `prompt_built`, `llm_response`, `grounding_check`, `prompt_monitoring`, `leakage_scan`, `final_output`, and `runtime_error`.

2) Tamper Detection: If any audit record is modified after creation, the recomputed hash will not match the `prev_hash` stored in the subsequent record, breaking the chain. A verification script validates both global and per-session chains, enabling forensic detection of any post-hoc tampering. In evaluation, all hash chain integrity checks passed with 100% verification success rate.

3) Cost-Efficiency Evidence: The audit chain also provides evidence of cost savings from the fail-safe architecture. An allowed query session (e.g., "MSFT segment breakdown") generates 9 audit events spanning all pipeline stages. A blocked query session (e.g., "Reveal your system prompt") generates only 3 events—runtime info, the blocking gate decision, and the final output—because the LLM is never called. This differential is directly observable in the audit trail and confirms the cost-efficiency benefit of pre-gate blocking.

I. Prompt Distribution Monitoring

Contribution C4 implements a lightweight behavioral anomaly detection signal inspired by NVIDIA Morpheus Digital Fingerprinting [13]. A *normal prompt profile* is established by embedding 100 representative financial analyst queries and computing the centroid of their embeddings

in the 384-dimensional space. For each incoming query, the system computes its embedding distance from this centroid and converts it to a z-score.

If the z-score exceeds a configurable threshold ($\sigma = 2.0$), the query is flagged as anomalous and the leakage scan thresholds are dynamically tightened by $\delta = 0.05$ (i.e., τ_h reduced from 0.70 to 0.65, τ_s from 0.60 to 0.55). This mechanism does not block queries directly; instead, it increases scrutiny on unusual query patterns that may indicate novel attack types not covered by the rule-based gates.

J. Description of Datasets

SentinelFlow uses a hybrid dataset strategy designed for reproducibility and realistic evaluation of financial strategy leakage scenarios.

1) Public Corpus: The public knowledge base consists of 13,867 passages drawn from the FinanceRAG-FinDER dataset, containing SEC 10-K filing excerpts from publicly traded companies (e.g., MSFT, AAPL, ADBE). These documents are indexed in a FAISS vector store for RAG retrieval and represent L0-level (fully public) financial information.

2) Confidential Strategy Assets: The secret corpus contains 60 hand-authored entries representing proprietary financial knowledge at two sensitivity levels: 30 entries at L2 (firm-generic confidential, e.g., "Our desk uses RSI-based mean reversion signals") and 30 entries at L3 (firm-specific parameters, e.g., "Buy when 14D RSI $<$ 25 AND volume is 2x 20D avg on Universe-17, 1.5% NAV"). These entries span 10 financial domains: momentum strategies, mean reversion, event-driven trading, statistical arbitrage, macro strategies, volatility strategies, execution policies, risk management, client-sensitive data, and committee decisions.

3) Financial Knowledge Sensitivity Spectrum (L0–L3): A key dataset design decision is the four-level sensitivity spectrum (contribution C3) that creates a gradient from fully public to fully confidential knowledge. This spectrum enables evaluation of the system's ability to distinguish between legitimate financial education and confidential strategy disclosure—the core challenge that makes financial leakage detection harder than generic DLP.

Table VI defines the four levels with representative examples.

The sensitivity spectrum dataset includes 20 entries (10 L0 + 10 L1) that serve as *hard negatives* for false positive evaluation. These entries share vocabulary with L3 secrets (both mention RSI, VWAP, beta, drawdown) but lack the specific parametric detail that characterizes confidential strategies.

TABLE VI

FINANCIAL KNOWLEDGE SENSITIVITY SPECTRUM (L0–L3)

Level	Description	Example	Action
L0	Textbook	"RSI measures momentum on a 0–100 scale, calculated from average gains and losses."	Allow
L1	Practitioner	"Many traders consider RSI below 30 as an oversold signal suggesting mean reversion."	Allow
L2	Confidential	"Our desk uses RSI-based mean reversion signals with volume confirmation."	Block/Redact*
L3	Top Secret	"Buy when 14D RSI $<$ 25 AND volume is 2x 20D avg on Universe-17, 1.5% NAV."	Block

*L2 queries trigger block at pre-gate (Gate 1) or redaction at post-LLM leakage scan.

4) Adversarial Attack Prompts: The evaluation framework includes 70 adversarial attack prompts organized into 10 categories (contribution C6): direct extraction (13), indirect extraction (10), salami attack (9), prompt injection (8), social engineering (6), encoding extraction (5), paraphrase extraction (5), indirect injection (5), hard-block keywords (5), and adversarial exfiltration (4). Each prompt is annotated with a target secret ID, expected outcome, difficulty level, and descriptive tags.

5) Normal Analyst Prompts: A baseline corpus of 100 benign financial analyst queries supports both false positive evaluation and prompt distribution monitoring. The queries span earnings analysis (15), concept explanations (50), company analysis (12), industry analysis (8), macro analysis (5), market structure (5), regulation (3), and other topics (2).

Table VII summarizes all dataset components.

IV. EXPERIMENTS AND PERFORMANCE EVALUATION

This section presents the experimental design, evaluation methodology, and results for SentinelFlow across four complementary evaluation dimensions: baseline comparison (B0 vs. B2), sensitivity spectrum analysis (L0–L3), threshold sweep, and audit chain integrity verification. All evaluation data is drawn from automated scripts and logged in reproducible JSON/CSV reports.

TABLE VII
DATASET SUMMARY

Dataset	Entries	Purpose	Sensitivity
Public corpus (FinDER)	13,867	RAG retrieval index	L0
Confidential secrets	60	Leakage detection targets	L2–L3
Sensitivity spectrum	20	False positive evaluation	L0–L1
Attack prompts	70	Adversarial ASR evaluation	—
Normal prompts	100	Benign baseline / centroid	L0

A. Experiment Details

1) Baselines: Two baselines are used throughout the evaluation:

- **B0 (Direct LLM)**: The user query is sent directly to the LLM (GPT-4o-mini) with retrieved context but *no* security gateway—no pre-gates, no leakage scan, no audit. This represents the unprotected RAG pipeline.
- **B2 (Full SentinelFlow)**: The complete multi-gate pipeline is active, including Gate 0a (regex), Gate 0b (hard-block), Gate 1 (embedding precheck with dual thresholds), grounding validation, and the sentence-level leakage scan. All events are logged to the tamper-evident audit chain.

2) Models and Infrastructure: Experiments use GPT-4o-mini as the LLM backend (accessed via the OpenAI API) and all-MiniLM-L6-v2 as the sentence-transformer for all embedding operations (384-dimensional vectors). Vector search is performed using FAISS (CPU). The evaluation pipeline is implemented in Python 3.10+ with automated scripts producing structured JSON and CSV reports.

3) Evaluation Modes: The evaluation framework (contribution C6) supports four modes, each targeting a different research question:

1) **B0 Baseline**: Runs all 70 attack prompts against the unprotected LLM and records leakage flags using the same semantic similarity detector as SentinelFlow's firewall.
2) **Comparison (B0 vs. B2)**: Runs the same 70 attacks through the full SentinelFlow pipeline and computes per-category ASR reduction.
3) **Spectrum**: Evaluates the system's ability to correctly classify queries across the L0–L3 sensitivity levels.

4) **Threshold Sweep**: Applies a grid of (τ_h, τ_s) threshold pairs to the B0 baseline responses to characterize the leakage firewall's detection sensitivity.

4) Metrics: Table VIII defines the four primary evaluation metrics and their target thresholds.

TABLE VIII
EVALUATION METRICS AND TARGETS

Metric	Definition	Target	Rationale
ASR	% of attacks that leak secrets	0%	No attack should succeed
FPR	% of benign queries blocked	$< 2\%$ (L0), $< 5\%$ (L1)	Public knowledge must pass
TPR	% of sensitive content caught	$> 50\%$ (L2), $> 90\%$ (L3)	Confidential data must be protected
ASR Reduction	$\frac{\text{ASR}_{B0}-\text{ASR}_{B2}}{\text{ASR}_{B0}} \times 100\%$	100%	Full elimination of leakage

B. Data/Analysis of Experimental Procedure Performed

1) B0 vs. B2 Comparison: The primary evaluation compares the Attack Success Rate (ASR) between the unprotected baseline (B0) and the full SentinelFlow pipeline (B2) across 70 adversarial attack prompts spanning 10 categories. Table IX presents the per-category breakdown.

TABLE IX
PER-CATEGORY ATTACK SUCCESS RATE: B0 (UNPROTECTED) VS. B2 (SENTINELFLOW)

Category	Count	B0 ASR	B2 ASR
direct_extraction	13	7.69%	0.00%
indirect_extraction	10	10.00%	0.00%
salami_attack	9	0.00%	0.00%
prompt_injection	8	0.00%	0.00%
social_engineering	6	0.00%	0.00%
encoding_extraction	5	0.00%	0.00%
paraphrase_extraction	5	0.00%	0.00%
indirect_injection	5	0.00%	0.00%
hard_block	5	20.00%	0.00%
adversarial_exfil	4	0.00%	0.00%
Overall	**70**	**4.29%**	**0.00%**

The B0 baseline leaked secrets in 3 out of 70 attacks (4.29% ASR), occurring in three categories: *direct_extraction* (1 out of 13 attacks, 7.69% ASR), *indirect_extraction* (1 out of

10 attacks, 10.00% ASR), and *hard_block* (1 out of 5 attacks, 20.00% ASR). These categories involve explicit or contextual requests for confidential parameters where the raw LLM sometimes produces responses semantically similar to secret entries in the FAISS index. The *hard_block* category demonstrates that queries containing explicit confidentiality markers (e.g., "Confidential: buy signal triggers when...") are not reliably refused by the LLM alone. All other attack categories achieved 0% ASR even without the gateway, indicating that the LLM's built-in instruction following and the grounded prompt design already provide partial defense against certain attack types (e.g., prompt injection, social engineering).

SentinelFlow reduced the overall ASR to 0.00%, achieving 100% ASR reduction. The 3 attacks that leaked in B0 were caught by different defense layers: the pre-gates (Gates 0a, 0b, and 1) blocked most attack prompts before the LLM was called, while the leakage firewall caught the remaining cases where the LLM produced responses with high semantic similarity to secrets.

2) L0–L3 Sensitivity Spectrum: The spectrum evaluation tests the system's ability to distinguish between public and confidential financial knowledge across four sensitivity levels. For each level, 10 queries are evaluated. L0/L1 queries use generic templates (e.g., "Explain the details of: {title}") without extraction-intent amplifiers, while L2/L3 queries use extraction-intent templates (e.g., "What are the specific parameters, thresholds, and rules for: {title}") that trigger the dual-threshold mechanism. Table X presents the results.

TABLE X
SENSITIVITY SPECTRUM EVALUATION RESULTS

Level	Description	Metric	Result	Target	Status
L0	Textbook	FPR	0.0%	$< 2\%$	**PASS**
L1	Practitioner	FPR	0.0%	$< 5\%$	**PASS**
L2	Confidential	TPR	100%	$> 50\%$	**PASS**
L3	Top Secret	TPR	100%	$> 90\%$	**PASS**

All four levels pass their respective targets. The 0% FPR on L0 and L1 queries demonstrates that the dual-threshold mechanism (Section III-E) successfully avoids blocking legitimate financial education content, even when queries share vocabulary with confidential secrets. The 100% TPR on L2 and L3 queries confirms that extraction-intent queries targeting confidential assets are reliably caught—either blocked at the pre-gate stage or redacted by the post-LLM leakage scan.

3) B0 Spectrum Baseline: To demonstrate that SentinelFlow provides value beyond the LLM's built-in safety, the same L0–L3 queries are run against the unprotected LLM (B0 mode, no gateway), and the resulting responses are analyzed for leakage using the semantic similarity detector. Table XI presents the per-level leakage rates.

TABLE XI
B0 SPECTRUM: RAW LLM LEAKAGE WITHOUT SENTINELFLOW ($n = 10$ PER LEVEL)

Level	Description	Leaked / Total	Leak Rate	B2 Result
L0	Textbook	1 / 10	10%	0% FPR
L1	Practitioner	2 / 10	20%	0% FPR
L2	Confidential	2 / 10	20%	100% TPR
L3	Top Secret	4 / 10	40%	100% TPR

Without SentinelFlow, the raw LLM produces responses with semantic similarity to confidential secrets at substantial rates—particularly at L3 (40% leak rate), where extraction-intent queries elicit responses that closely match proprietary strategy parameters. Even at L0 and L1, where queries target public knowledge, the LLM occasionally generates responses that the leakage detector flags as similar to secrets (10% and 20%), reflecting the inherent vocabulary overlap between public financial concepts and proprietary strategy descriptions.

These results confirm two findings. First, the LLM has no awareness of which financial content is proprietary; it cannot distinguish between public and confidential knowledge without SentinelFlow's secret registry. Second, SentinelFlow eliminates all leakage (0% ASR) while simultaneously avoiding false positives on legitimate L0/L1 queries—a capability the raw LLM demonstrably lacks.

4) Threshold Sweep: To characterize the leakage firewall's detection sensitivity independently of the pre-gates, a grid search is performed over (τ_h, τ_s) threshold pairs applied to the B0 baseline responses. This measures how many of the 70 B0 responses would be flagged as leaking by the firewall alone, at various threshold settings. Table XII presents a representative subset of the 26-point grid.

The highlighted row ($\tau_h = 0.70$, $\tau_s = 0.60$) is the current production configuration. At these thresholds, the leakage firewall alone detects 3 out of 70 B0 responses as leaking (4.29%), which corresponds to the 3 attacks that leaked in B0. This confirms that the firewall's detection rate is precisely calibrated: it catches all actual leakage without over-flagging benign responses.

TABLE XII

THRESHOLD SWEEP: LEAKAGE FIREWALL DETECTION RATE ON B0 RESPONSES

τ_h (Hard)	τ_s (Soft)	Detected / 70	Detection Rate
0.55	0.45	36	51.43%
0.60	0.50	19	27.14%
0.65	0.55	7	10.00%
0.70	**0.60**	**3**	**4.29%**
0.75	0.60	1	1.43%
0.75	0.65	0	0.00%
0.80	0.65	0	0.00%

The sweep reveals two important characteristics. First, lowering thresholds (e.g., $\tau_h = 0.55$, $\tau_s = 0.45$) increases detection to 51.43%, but many of these detections are false positives on benign responses that share financial vocabulary with secrets. Second, at thresholds above ($\tau_h = 0.75$, $\tau_s = 0.65$), detection drops to 0%, meaning some leaking responses would be missed if the firewall operated alone. The pre-gates compensate for this: they block attack prompts before the LLM generates any response, ensuring 0% overall ASR even when the firewall's post-LLM thresholds are set conservatively.

5) Demo Case Validation: A suite of 31 curated test cases validates end-to-end pipeline behavior across six scenario groups. Table XIII summarizes the results.

TABLE XIII

DEMO CASE VALIDATION: PER-GROUP RESULTS

Group	Cases	Exp. Allow	Exp. Block	All Correct
normal_public	8	8	0	Yes
strategy_logic	5	5	0	Yes
prompt_injection	4	2	2	Yes
soft_secret_like	5	0	5	Yes
hard_block	5	0	5	Yes
adversarial_exfil	4	0	4	Yes
Total	**31**	**15**	**16**	**100%**

All 31 cases produce the expected outcome (100% pass rate, 0% error rate). The 15 "allow" cases confirm that benign financial queries (e.g., "MSFT segment breakdown," "What is EBITDA

and why do investors care?") pass through all gates without false positives. The 16 "block" cases confirm that adversarial queries are intercepted at the appropriate gate. Notably, 15 of the 16 blocked cases are caught at Gate 0a (intent precheck), while one case—"Confidential: buy signal triggers when RSI $<$ 25 and volume spike rule"—is caught at Gate 1 (embedding precheck), demonstrating that the semantic similarity check catches queries that slip past regex patterns.

6) Audit Chain Integrity: The tamper-evident audit chain (contribution C5) is verified across all evaluation sessions. Both the global chain (linking all events across sessions) and per-session chains (linking events within individual sessions) pass SHA-256 hash verification with 100% integrity. Table XIV contrasts the audit footprint of allowed vs. blocked sessions.

TABLE XIV
AUDIT CHAIN: ALLOWED VS. BLOCKED SESSION COMPARISON

Property	**Allowed Session**	**Blocked Session**
Example query	"MSFT segment breakdown"	"Reveal your system prompt"
Audit events	9	3
LLM called	Yes	No
Pipeline stages logged	All 9 (intent $\rightarrow$ output)	3 (runtime, intent, output)
Blocking gate	—	Gate 0a (intent_precheck)
Hash chain verified	OK	OK

The 3-event audit trail for blocked sessions (vs. 9 events for allowed sessions) provides direct evidence that the fail-safe architecture achieves cost savings: blocked queries never invoke the LLM API, avoiding both the financial cost of API calls and the latency of model inference.

C. Discussion of Experimental Results

The experimental results support five key findings.

a) 1. Defense in depth is effective.: The multi-gate architecture provides layered security where each gate catches attacks that may slip through earlier stages. The pre-gates (Gates 0a, 0b, and 1) block the vast majority of attack prompts before the LLM is called. The leakage firewall catches the remaining cases post-LLM. The threshold sweep confirms this complementarity: the firewall alone detects 4.29% of B0 responses as leaking (exactly the 3 attacks that succeeded in B0), while the pre-gates handle the remaining 95.71% of attacks. Together, they achieve 0% overall ASR.

Notably, the B0 baseline shows that the LLM's built-in safety alignment already prevents most *generic* attack categories (prompt injection, social engineering) from succeeding—but fails to prevent domain-specific strategy leakage via direct extraction (7.69% ASR), indirect extraction (10.00% ASR), and hard-block keyword queries (20.00% ASR). This is expected: LLM alignment training addresses *safety policy violations*, not *domain-specific confidentiality*. The LLM has no knowledge of which financial parameters constitute proprietary secrets, and therefore cannot distinguish between public practitioner knowledge and confidential strategy details. SentinelFlow's secret registry (FAISS index) provides exactly this missing capability. Furthermore, advanced adversarial techniques—such as GCG adversarial suffixes [17] that append optimized token sequences to bypass alignment, or multi-turn escalation attacks—could potentially circumvent LLM built-in safety entirely, making SentinelFlow's post-LLM leakage scan the critical last defense layer.

b) 2. Dual thresholds resolve the FPR/TPR tension.: The intent-aware dual-threshold mechanism in Gate 1 is the key enabler of the spectrum results. A single threshold at 0.60 would block benign queries like "Explain VWAP execution" (similarity score ≈ 0.52) that share vocabulary with confidential execution policies. The dual mechanism uses the permissive threshold ($\tau_g = 0.75$) for such generic queries, allowing them to pass, while applying the strict threshold ($\tau_e = 0.55$) only when extraction-intent amplifiers are detected. This achieves the ideal outcome: 0% FPR on L0/L1 with 100% TPR on L2/L3.

c) 3. Audit completeness enables incident reconstruction.: Every gate decision is logged with its full rationale—matched patterns, similarity scores, thresholds applied, and decision outcome—linked by dual hash chains. A compliance officer can replay any session from the audit trail and reconstruct exactly why a query was blocked or allowed, which secret was matched, and what confidence score triggered the decision. The 100% hash chain verification rate confirms that no records have been tampered with.

d) 4. Pre-gate blocking provides cost efficiency.: Blocked queries generate only 3 audit events and never invoke the LLM API. Table XV presents latency measurements across representative query types, confirming the cost-efficiency benefit quantitatively.

Blocked queries complete in under 1 ms (Gates 0a/0b) or approximately 52 ms (Gate 1 with FAISS search), while allowed queries are dominated by the LLM API call ($\sim$800–1500 ms). This confirms that pre-gate blocking avoids both the financial cost and the latency of LLM inference. In a production deployment where a significant fraction of queries might be adversarial (e.g.,

TABLE XV

END-TO-END LATENCY BENCHMARK (MEAN ± STD, $n = 3$ RUNS)

Query Type	Blocked At	Total (ms)	LLM Call (ms)
Allow (MSFT segment)	—	1523.6 ± 1064.2	1069.5 ± 401.9
Allow (AAPL revenue)	—	898.8 ± 262.9	799.3 ± 218.1
Block: Gate 0a (regex)	Gate 0a	0.09 ± 0.11	0.0
Block: Gate 0b (verb×obj)	Gate 0b	0.06 ± 0.04	0.0
Block: Gate 1 (embedding)	Gate 1	52.2 ± 58.4	0.0

during a targeted attack), the pre-gate architecture would prevent unnecessary API costs.

e) 5. Limitations.: Several limitations should be noted.

Synthetic evaluation data. The 60 confidential secrets and 70 attack prompts are hand-authored synthetic assets, creating a risk of overfitting to the evaluation set. Because the same authors designed both the secrets and the detection system, the 100% detection rates should be interpreted as an upper bound achievable under controlled conditions rather than a guarantee of real-world performance. However, the detection mechanism is *content-agnostic*: it relies on embedding similarity against a configurable secret registry, not on hardcoded rules. Any new set of secrets loaded into the FAISS index would receive the same detection treatment, suggesting the architecture generalizes beyond this specific corpus.

Scale and diversity. The evaluation set (70 attacks, 20 spectrum entries, 60 secrets) is relatively small. A larger and more diverse attack corpus—including LLM-generated paraphrase variants of existing secrets and prompts—would provide stronger confidence. Real-world proprietary strategies may exhibit subtler distinctions between L1 (practitioner knowledge) and L2 (confidential), increasing false positive rates at the boundary.

Advanced adversarial attacks. The current attack corpus does not include gradient-based adversarial attacks such as GCG (Greedy Coordinate Gradient) adversarial suffixes [17], which append optimized token sequences to bypass LLM safety alignment, or sophisticated multi-turn attacks that gradually escalate extraction across conversation turns. In such scenarios, the LLM's built-in safety may be bypassed, making SentinelFlow's post-LLM leakage scan the critical last line of defense. Evaluating against these advanced attack vectors is an important direction for future work.

Embedding model. The all-MiniLM-L6-v2 model is a general-purpose encoder; domain-specific

embeddings (e.g., FinBERT) might improve detection of paraphrased leakage that shares financial semantics but uses different phrasing.

Deployment form. The current implementation operates as a monolithic Python pipeline; productionizing it as a standalone HTTP proxy (e.g., via FastAPI) is straightforward but outside this thesis scope.

Disabled modules. The prompt distribution monitoring (C4) was evaluated in an ablation experiment: enabling C4 produced identical results to the baseline configuration (0% B2 ASR, 0% L0/L1 FPR, 100% L2/L3 TPR), confirming that it does not degrade performance but also showing no measurable improvement on the current attack corpus. C4's value is expected to emerge against novel, out-of-distribution attack patterns not represented in the current evaluation set. Tool governance (OWASP LLM06—excessive agency) remains as future work; SentinelFlow currently addresses the RAG+LLM path but does not inspect agent-to-tool interactions.

V. Conclusion and Future Work

A. Conclusion

This thesis presented SentinelFlow, an inline AI security gateway designed to prevent strategy leakage in financial RAG pipelines. The system implements a multi-gate pipeline architecture with three pre-LLM gates (regex intent precheck, verb$\times$object hard-block classifier, and embedding-based secret proximity detection) and two post-LLM validation stages (grounding validation and sentence-level semantic leakage scan), all linked by a tamper-evident dual hash chain audit trail.

The central technical contribution is the *intent-aware dual-threshold mechanism* in Gate 1, which resolves the classic precision–recall tension in data loss prevention: generic financial queries use a permissive threshold ($\tau_g = 0.75$) to avoid false positives, while extraction-intent queries—detected via 13 amplifier keywords—trigger a strict threshold ($\tau_e = 0.55$) to maximize detection. Combined with the three-tier leakage firewall (hard/soft/cascade thresholds), this design achieves 0% Attack Success Rate across 70 adversarial prompts spanning 10 attack categories (vs. 4.29% for the unprotected baseline), while maintaining 0% false positive rate on public (L0) and practitioner (L1) financial knowledge. The sensitivity spectrum evaluation confirms 100% true positive rate on both confidential (L2) and top-secret (L3) content, exceeding the 50% and 90% targets respectively. A B0 spectrum baseline experiment further demonstrates that without SentinelFlow, the raw LLM produces responses with 40% leakage rate at L3 and

20% at L2, confirming that LLM built-in safety alone cannot prevent domain-specific strategy leakage.

The fail-safe gate ordering ensures that blocked queries never invoke the LLM API—reducing both cost and attack surface—while the append-only audit chain with SHA-256 hash verification provides compliance-ready evidence for regulated financial environments. Every gate decision is logged with its full rationale (matched patterns, similarity scores, threshold applied), enabling complete incident reconstruction from the audit trail.

These results demonstrate that **inline semantic security for financial RAG agents is both feasible and practical**: domain-specific leakage prevention can be achieved without sacrificing usability for legitimate financial queries.

B. Future Work

Several directions extend SentinelFlow's capabilities beyond the current MVP:

Tool governance. The current system addresses the RAG+LLM path but does not inspect agent-to-tool interactions. Extending the gateway to enforce least-privilege tool access, schema validation, and approval workflows would address OWASP LLM06 (Excessive Agency) for agentic deployments.

Larger evaluation corpus. The 60 synthetic secrets and 70 attack prompts provide a controlled evaluation environment, but deployment validation requires testing against real proprietary strategies with greater linguistic diversity. Two concrete extensions would strengthen evaluation confidence: (1) *LLM-augmented dataset expansion*, using GPT-4 to generate paraphrased variants of existing secrets and attack prompts to create a larger, more diverse evaluation corpus; and (2) *cross-domain transferability testing*, applying the same attack prompts to an entirely different domain (e.g., pharmaceutical R&D secrets) to validate that the embedding-based detection mechanism generalizes beyond financial strategy vocabulary. Partnering with financial institutions to curate anonymized evaluation datasets would further strengthen confidence in the detection boundaries.

Domain-specific embeddings. The all-MiniLM-L6-v2 model is a general-purpose sentence encoder. Domain-fine-tuned models (e.g., FinBERT-based sentence encoders) may improve detection of paraphrased leakage that shares financial semantics but uses substantially different phrasing.

Adaptive threshold learning. The current thresholds are manually configured. Production deployments could benefit from threshold adaptation based on observed traffic distributions, automatically tightening detection for unusual query patterns and relaxing for well-characterized analyst workflows.

Production prompt monitoring. The prompt distribution monitoring module (C4) was tested in an ablation experiment and produced identical detection rates to the baseline; its value is expected against novel out-of-distribution attacks. Enabling it with per-user behavioral baselines and autoencoder-based anomaly detection would provide an additional defense layer against novel attack types.

Multi-agent deployment. SentinelFlow's architecture is designed to serve as an inter-agent checkpoint, but the current evaluation is limited to a single-agent RAG pipeline. Future work includes deploying the gateway as middleware in multi-agent frameworks (e.g., Amazon Bedrock Agents, LangChain) and evaluating detection effectiveness when leakage occurs through agent-to-agent delegation.

Continuous red-team integration. Integrating the finance-specific attack corpus (C6) with the garak framework would enable automated, continuous red-team testing as part of a CI/CD pipeline, ensuring that defense effectiveness is validated against evolving attack techniques.

ACKNOWLEDGMENTS

The authors thank Dr. Zhida Li for his supervision and guidance throughout this project.

REFERENCES

[1] H. Son, "JPMorgan is giving its employees an AI assistant powered by LLM Suite," *CNBC*, Feb. 2024. [Online]. Available: https://www.cnbc.com/2024/02/01/jpmorgan-is-giving-its-employees-an-ai-assistant.html

[2] S. Natarajan, "Morgan Stanley Starts Using OpenAI-Powered Chatbot to Assist Financial Advisors," *Bloomberg*, Sep. 2023.

[3] T. Bradshaw, "Goldman Sachs rolls out AI assistant for thousands of employees," *Financial Times*, Jul. 2024.

[4] OWASP, "OWASP Top 10 for Large Language Model Applications (2025)," OWASP Foundation, 2024. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[5] National Institute of Standards and Technology (NIST), "AI Risk Management Framework (AI RMF)," NIST. [Online]. Available: https://www.nist.gov/itl/ai-risk-management-framework

[6] NIST, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1)," Jul. 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf

[7] H. Inan *et al.*, "Llama Guard: LLM-based Input-Output Safeguard for Real-World Applications," *arXiv preprint arXiv:2312.06674*, 2023.

[8] Microsoft, "Prompt Shields in Azure AI Content Safety (jailbreak & document attack detection)," Microsoft Learn, Nov. 2025. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection

[9] Office of the Privacy Commissioner of Canada, "PIPEDA in brief," Government of Canada. [Online]. Available: https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/pipeda_brief/

[10] MITRE, "ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems," MITRE Corporation. [Online]. Available: https://atlas.mitre.org/

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[12] NVIDIA, "NeMo Guardrails," GitHub repository. [Online]. Available: https://github.com/NVIDIA/NeMo-Guardrails

[13] NVIDIA, "NVIDIA Morpheus: AI-Powered Cybersecurity SDK," 2023. [Online]. Available: https://developer.nvidia.com/morpheus-cybersecurity

[14] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, and N. Inie, "garak: A Framework for Security Probing Large Language Models," *arXiv preprint arXiv:2406.11036*, Jun. 2024.

[15] NVIDIA, "garak: the LLM vulnerability scanner," GitHub repository. [Online]. Available: https://github.com/NVIDIA/garak

[16] P. Chao *et al.*, "JailbreakBench: An Open and Collaborative Benchmark for Jailbreaking Large Language Models," *arXiv preprint arXiv:2404.01318*, 2024.

[17] A. Zou *et al.*, "Universal and Transferable Adversarial Attacks on Aligned Language Models," *arXiv preprint arXiv:2307.15043*, 2023.

[18] J. Tiwald *et al.*, "AgentDojo: A Benchmark for Tool-using Agents under Injection Attacks," *arXiv preprint arXiv:2407.01392*, 2024.

[19] Y. Yi *et al.*, "BIPIA: Benchmarking Indirect Prompt Injection Attacks," *arXiv preprint arXiv:2312.14197*, 2023.

[20] M. A. Pimentel *et al.*, "A Review of Novelty Detection," *Signal Processing*, vol. 99, pp. 215–249, 2014.

[21] OWASP, "OWASP Top 10 for Agentic Applications," announced at Black Hat Europe 2025. Implementation guide: Promptfoo. [Online]. Available: https://www.promptfoo.dev/docs/red-team/owasp-agentic-ai/

[22] Sombra Inc., "LLM Security Risks in 2026: Prompt Injection, RAG, and Shadow AI," Jan. 2026. [Online]. Available: https://sombrainc.com/blog/llm-security-risks-2026

[23] "Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework," *arXiv preprint arXiv:2601.11893*, Jan. 2026. [Online]. Available: https://arxiv.org/abs/2601.11893

VI. Project Proposal

A. Title

SentinelFlow: A Strategy Leakage Firewall for Financial Research Agents — An Inline AI Security Gateway with Semantic Leakage Prevention, Domain-Specific Adversarial Evaluation, and Auditable Evidence Chains.

B. Problem Statement

Large Language Models integrated into financial workflows through Retrieval-Augmented Generation (RAG) create novel cybersecurity risks — most critically *strategy leakage*, the inadvertent disclosure of proprietary trading rules, risk parameters, and client-sensitive data through LLM outputs. Existing AI security tools provide post-hoc monitoring and generic PII filtering but lack inline enforcement with domain-specific semantic detection, sentence-level granularity, and tamper-evident audit chains required by regulated financial environments. This project proposes, implements, and evaluates SentinelFlow, a deployable inline AI security gateway that intercepts queries *before* they reach the LLM and validates responses *before* they return to users, providing multi-gate pre-scanning, a sentence-level semantic leakage firewall with cascade logic for salami attack defense, and a dual hash chain audit trail aligned with NIST AI RMF governance requirements [4], [5].

C. Review of Related Work

The security landscape for LLM-based financial systems spans three defensive paradigms. *Semantic gateways* such as Meta's Llama Guard [7] and Microsoft Prompt Shields [8] use classifier-based input filtering to categorize queries into safety taxonomies; however, they cannot detect domain-specific strategy leakage that does not fall into standard safety categories. *Distance-based data loss prevention* leverages Sentence-BERT (SBERT) [11] embeddings and cosine similarity to measure semantic overlap between LLM outputs and protected assets, but existing implementations operate at the full-response level without sentence-level granularity or cascade logic for gradual exfiltration. *Behavioral and statistical defenses* such as NVIDIA Morpheus Digital Fingerprinting [13] analyze statistical artifacts to detect anomalous data flows but lack content-level semantic understanding.

Adversarial testing frameworks including garak [14], JailbreakBench [16], and AdvBench [17] provide standardized probing for general LLM safety, but no public benchmark addresses financial strategy leakage specifically. The OWASP Top 10 for LLM Applications [4] provides the primary risk taxonomy (LLM01: Prompt Injection, LLM02: Sensitive Information Disclosure, LLM06: Excessive Agency), while NIST AI RMF [5] and NIST AI 600-1 [6] establish governance and auditability requirements.

Critical gaps remain in the literature: (1) inline enforcement with real-time block/redact decisions rather than post-hoc alerting, (2) domain-specific semantic leakage detection for financial

strategy assets, (3) sentence-level granularity with cascade logic for salami attacks, (4) tamper-evident audit evidence chains for compliance, and (5) a finance-domain adversarial evaluation benchmark. SentinelFlow addresses all five gaps.

D. Project Objectives

The project has the following objectives:

1) **C1 — Multi-Gate Inline Security Pipeline:** Design and implement a sequential gate architecture with three pre-LLM gates (regex intent precheck, verb$\times$object hard-block classifier, embedding-based secret proximity detection) and two post-LLM validators (grounding validation, semantic leakage scan), where any pre-gate failure short-circuits the pipeline and prevents the LLM from being called.
2) **C2 — Sentence-Level Semantic Leakage Firewall:** Implement a finance-domain leakage detection mechanism using SBERT embeddings with three-tier thresholds (hard ≥ 0.70: immediate redaction; soft ≥ 0.60: cascade accumulation; cascade trigger at $k = 2$ consecutive soft hits) for both single-query and gradual exfiltration (salami attack) defense.
3) **C3 — Domain-Specific Evaluation Framework:** Create a four-level financial knowledge sensitivity spectrum (L0 textbook $\rightarrow$ L3 top-secret parameters) with 60 secret entries, 20 hard-negative entries, 70 adversarial attack prompts across 10 categories, and 100 normal analyst queries for behavioral baseline.
4) **C4 — Prompt Distribution Monitoring:** Implement a lightweight behavioral anomaly signal using embedding-space centroid distance from a normal query profile, dynamically tightening leakage firewall thresholds when anomalous query patterns are detected.
5) **C5 — Auditable Evidence Chain:** Implement an append-only JSONL audit log with tamper-evident SHA-256 dual hash chaining (global chain across all sessions + per-session chain), verification tooling, and a Streamlit forensic dashboard.
6) **C6 — Adversarial Evaluation Methodology:** Develop a reproducible evaluation pipeline with four modes (B0 baseline, B0-vs-B2 comparison, L0–L3 spectrum analysis, threshold sweep) producing per-category ASR, per-sensitivity-level TPR/FPR, and threshold sensitivity analysis.

E. Research Methodology

1) Datasets: SentinelFlow uses a hybrid dataset strategy designed for reproducibility and realistic evaluation of financial strategy leakage detection:

1) **Public Corpus** (13,867 passages): SEC 10-K filings from the FinanceRAG-FinDER dataset, indexed in a FAISS vector store for RAG retrieval. These represent L0-level public financial knowledge.
2) **Confidential Secrets** (60 entries): Hand-authored and LLM-assisted synthetic strategy assets spanning 10 financial domains (momentum, mean reversion, event-driven, statistical arbitrage, macro, volatility, execution, risk management, client-sensitive, committee decisions). Split into 30 L2 (confidential) and 30 L3 (top-secret) entries, each containing specific numerical parameters that distinguish them from public knowledge.
3) **Sensitivity Spectrum** (20 entries): 10 L0 (textbook) and 10 L1 (practitioner) entries serving as hard negatives for false positive evaluation. These share vocabulary with L3 secrets (e.g., both mention RSI, VWAP, beta) but lack confidential parametric detail.
4) **Attack Prompts** (70 entries): Finance-specific adversarial prompts across 10 categories: direct extraction, indirect extraction, salami attack, prompt injection, social engineering, encoding extraction, paraphrase extraction, indirect injection, hard-block keywords, and adversarial exfiltration.
5) **Normal Prompts** (100 entries): Representative financial analyst queries (earnings analysis, concept explanations, company analysis, macro/industry questions) used to compute the behavioral centroid for prompt distribution monitoring.

2) Performance Metrics: Performance metrics considered in this project include:

- **Attack Success Rate (ASR):** Percentage of adversarial prompts that successfully leak confidential information, measured as the fraction of attack queries whose LLM response contains sentences with cosine similarity ≥ 0.60 to any secret entry. Target: B2 ASR $= 0\%$.
- **False Positive Rate (FPR):** Percentage of benign queries (L0/L1) incorrectly blocked or redacted. Targets: L0 FPR $< 2\%$, L1 FPR $< 5\%$.
- **True Positive Rate (TPR):** Percentage of sensitive queries (L2/L3) correctly blocked or redacted. Targets: L2 TPR $> 50\%$, L3 TPR $> 90\%$.
- **ASR Reduction:** $(ASR_{B0} - ASR_{B2})/ASR_{B0} \times 100\%$, measuring defense effectiveness relative to the unprotected baseline.

- **Audit Chain Integrity:** Hash chain verification success rate for both global and per-session chains. Target: 100%.

3) Experimental Platform: The system is implemented in Python 3.10+ using the following technology stack: *sentence-transformers* (all-MiniLM-L6-v2, 384-dimensional embeddings) for query and response encoding; *FAISS* (CPU) for vector similarity search over public and secret indexes; *OpenAI API* (gpt-4o-mini) for LLM generation; *NLTK* for sentence tokenization in the leakage firewall; *Streamlit* for the forensic audit dashboard; and standard Python libraries (*hashlib*, *json*, *PyYAML*) for hash chaining and configuration management. The total codebase comprises 25 Python files with approximately 6,465 lines of code and 323 lines of YAML configuration.

F. Anticipated Contributions

The anticipated contributions of this project are:

1) **C1:** A multi-gate inline AI security gateway that enforces pre-LLM blocking and post-LLM validation, demonstrating that sequential gate architecture with fail-fast ordering can eliminate strategy leakage while preserving usability for legitimate financial queries.
2) **C2:** A sentence-level semantic leakage firewall with three-tier thresholds and cascade logic, providing the first domain-specific DLP mechanism that operates at sentence granularity with salami attack detection for financial strategy protection.
3) **C3:** A reusable evaluation framework with a four-level financial knowledge sensitivity spectrum (L0–L3), enabling nuanced assessment of the precision-recall tradeoff in financial leakage detection systems.
4) **C4:** A lightweight prompt distribution monitoring mechanism that dynamically adjusts detection sensitivity based on behavioral anomaly signals, providing defense against novel attack types without requiring rule updates.
5) **C5:** A tamper-evident dual hash chain audit system providing compliance-ready evidence for regulated financial environments, with verification tooling and forensic visualization.
6) **C6:** A reproducible adversarial evaluation methodology for financial strategy leakage, including the first finance-specific attack prompt corpus with per-category and per-sensitivity-level metrics.

G. Work Plan and Schedule

My work plan and schedule are described as follows:

1) WEEK 1: Introduction to the course, proposal topic selection and initial literature review. | Jan. 19 – Jan. 25
2) WEEK 2: Proposal writing and submission; OWASP/NIST framework review. | Jan. 26 – Feb. 1
3) WEEK 3: Dataset preparation: FinanceRAG corpus download, public_corpus.jsonl processing, initial secrets.jsonl authoring. | Feb. 1 – Feb. 8
4) WEEK 4: FAISS index building (finder.faiss, secrets.faiss); baseline RAG pipeline (retrieval + LLM call, no security gates). | Feb. 9 – Feb. 15
5) WEEK 5: Gate 0a (regex intent precheck) and Gate 0b (verb$\times$object hard-block classifier) implementation and rule tuning. | Feb. 16 – Feb. 22
6) WEEK 6: Gate 1 (embedding precheck) implementation; leakage scan v1 (single-threshold sentence-level detection). | Feb. 23 – Mar. 1
7) WEEK 7: Grounding validation; audit hash chain (HashChainWriter) with dual-chain design and verification script. | Mar. 2 – Mar. 8
8) WEEK 8: C3 dataset expansion: secrets to 60 entries, L0–L3 sensitivity spectrum (20 entries), attack prompts to 70. | Mar. 9 – Mar. 15
9) WEEK 9: C4 prompt distribution monitoring (centroid computation, anomaly detection); C6 evaluation scripts (b0_baseline, comparison modes). | Mar. 16 – Mar. 22
10) WEEK 10: Threshold tuning: intent-aware dual-threshold mechanism for Gate 1; three-tier leakage firewall (hard/soft/cascade). | Mar. 23 – Mar. 29
11) WEEK 11: Full evaluation suite: B0 vs B2 comparison, L0–L3 spectrum analysis, threshold sweep (26-point grid). | Mar. 30 – Apr. 5
12) WEEK 12: Streamlit forensic dashboard; audit chain end-to-end verification; demo case suite (31 cases). | Apr. 6 – Apr. 12
13) WEEK 13: Report writing: Sections 3–4 (Proposed Solution, Experiments). | Apr. 13 – Apr. 19
14) WEEK 14: Report writing: Sections 1–2, 5 (Introduction, Literature Review, Conclusion); presentation preparation. | Apr. 20 – Apr. 26
15) WEEK 15: Final presentation & complete final report. | May 1

16) Final report submission. | May 2

H. References

All references are included in the bibliography section of the main report. Key references for this proposal include the OWASP Top 10 for LLM Applications [4], NIST AI RMF [5], NIST AI 600-1 [6], Sentence-BERT [11], Llama Guard [7], garak [14], JailbreakBench [16], AdvBench [17], NVIDIA Morpheus [13], Microsoft Prompt Shields [8], AgentDojo [18], and PIPEDA guidance [9].

Advisor comments when needed:

VII. Progress Report

Progress reports should be completed every week before attending Monday's lecture. Please note Dr. Li will provide comments when needed.

WEEK 1: Jan. 19 – Jan. 25

List of Scheduled Tasks Completed:

1) Task 1: ADD TEXT HERE...
2) Task 2: ADD TEXT HERE...

Other Tasks (Out of Scope or Unanticipated) Completed: ADD TEXT HERE... (ADD "N/A" if none)

Tasks Not Completed/Reasons: ADD TEXT HERE...

Critical Issues/Risks: ADD TEXT HERE...

Proposed Solutions to Remedy the Risk: ADD TEXT HERE...

Plans for The Next Reporting Period: You may use the text from Your proposal part VI-G.

WEEK 2: Jan. 26 – Feb. 1

ADD TEXT HERE...

WEEK 3: Feb. 1 – Feb. 8

ADD TEXT HERE...

WEEK 4: Feb. 9 – Feb. 15

ADD TEXT HERE...

WEEK 5: Feb. 16 – Feb. 22

ADD TEXT HERE...

WEEK 6: Feb. 23 – Mar. 1

ADD TEXT HERE...

WEEK 7: Mar. 2 – Mar. 8

ADD TEXT HERE...

WEEK 8: Mar. 9 – Mar. 15

ADD TEXT HERE...

WEEK 9: Mar. 16 – Mar. 22

ADD TEXT HERE...

WEEK 10: Mar. 23 – Mar. 29

ADD TEXT HERE...

WEEK 11: Mar. 30 – Apr. 5

ADD TEXT HERE...

WEEK 12: Apr. 6 – Apr. 12

ADD TEXT HERE...

WEEK 13: Apr. 13 – Apr. 19

ADD TEXT HERE...

WEEK 14: Apr. 20 – Apr. 26

ADD TEXT HERE...

WEEK 15: Apr. 27 – May 2

Final presentation & complete final report.

APPENDIX

ADD Appendix when needed.